

Teach useful behavior.

Learn from examples and choices.

See how demonstrations and preferences guide learning, then inspect what adaptation actually changes.

🕒 20-25 min core reading · 20-30 min practice · 12 short sections

The movie assistant has the right evidence but repeatedly writes unsupported, overconfident recommendations. You have tested a clear prompt and now want to teach a more reliable response pattern. Supervised fine-tuning learns from demonstrations; preference learning uses comparisons between answers. Both depend on what your data actually rewards.

WHAT YOU WILL UNDERSTAND

The main question: which training signal changes which parameters, and how will you know the change helped?

The practical path: build demonstrations, inspect a low-rank update, compare reward-based learning with DPO, and evaluate a candidate before deployment.

Reading path: twelve short sections, three diagrams, two numerical experiments, and a Python workbook. Expand the optional notes when you want the underlying mechanics.

1. Define the behavior you want to teach

Start with an observable target: “Use the supplied movie evidence, respect constraints, and state when availability is unknown.” “Make the model smarter” is too vague to turn into labels or an evaluation.

Separate **domain adaptation** from **instruction adaptation**. Continuing language-model training on domain text can teach its vocabulary and patterns; supervised instruction examples teach how to respond to tasks. Neither guarantees reliable recall of an exact changing catalog. Keep retrieval and live tools when the application needs them.

Choose a suitable base checkpoint and record a prompt baseline. Define success cases, difficult cases, and behaviors you must preserve. The same training method can help one task and harm another, so a lower training loss is only one signal.

2. Build demonstrations that match real requests

In **supervised fine-tuning (SFT)**, each example provides an input and a desired output. A movie example should contain the evidence available at deployment, not hidden facts the model will never receive.

```
Input: Recommend a calm film under 100 minutes.  
Evidence: M001, Moonlight Map, 94 minutes, gentle pacing.  
Streaming availability: not supplied.  
Target: Moonlight Map fits the runtime limit and calm mood.  
        I cannot establish streaming availability from this card.
```

Include realistic variety: no matches, conflicting constraints, missing evidence, alternate wording, and valid tool failures. Review targets for unsupported claims and consistent formatting. A hundred near-duplicates do not provide the coverage of a hundred meaningfully different requests.

Split data before expanding it with paraphrases. Keep related conversations or movie examples together when needed to avoid leakage. Use a training set for updates, a validation set for choices such as stopping, and a final test set reserved for assessment. Check permission to use the data and remove secrets or unnecessary personal information.

➤ **Demonstrations teach a target. Preferences compare targets.**

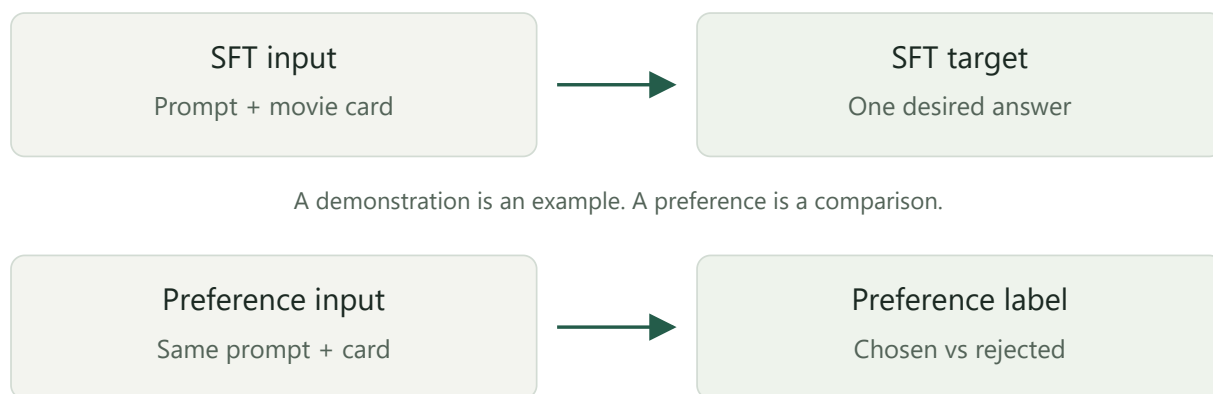


Figure 1. SFT trains on a desired completion for a prompt. Preference learning compares two completions under the same prompt and rubric. Both change behavior through optimization; neither makes an unsupported label true.

▼ **Go deeper: chat templates, truncation, and synthetic examples**

Chat models expect specific role markers and end-of-turn tokens. Use the checkpoint's supported chat template consistently in training and inference. Verify the tokenized result rather than assuming a list of messages automatically becomes the right sequence.

Inspect examples after truncation: cutting away a crucial movie constraint while keeping its answer teaches a misleading mapping. Deduplicate near-identical examples and keep augmented versions within the same split.

Synthetic demonstrations can expand coverage, but a generating model can repeat its own errors. Apply the same evidence and quality checks as for human examples; preserve provenance and manually audit representative slices. Quantity alone does not repair a bad target definition.

3. Understand what SFT optimizes

For an autoregressive model, SFT usually increases the probability of target tokens given the prompt and preceding target tokens. This use of the known target prefix is **teacher forcing**. A common assistant-only objective masks prompt tokens out of the loss:

$$L_{SFT} = -\frac{1}{\sum_t m_t} \sum_t m_t \log p_{\theta}(y_t \mid x, y_{<t}).$$

x is the prompt and evidence, y_t a target token, and m_t is 1 for positions included in the objective and 0 otherwise. The denominator counts included tokens; an example with none should be rejected. Training then uses backpropagation and an optimizer to update the selected parameters.

A loss mask is not an attention mask. Masking the user's words out of the loss does not hide them from the assistant. They remain context for predicting its answer. Some recipes train on more tokens; inspect your configuration rather than assuming one universal default. See the [SFT trainer documentation](#) for supported loss and dataset choices.

If a target token has probability 0.25, its negative log-likelihood is about 1.3863 nats. Raising that probability to 0.5 reduces the contribution to 0.6931. This rewards matching the target, even when the target itself contains a factual error.

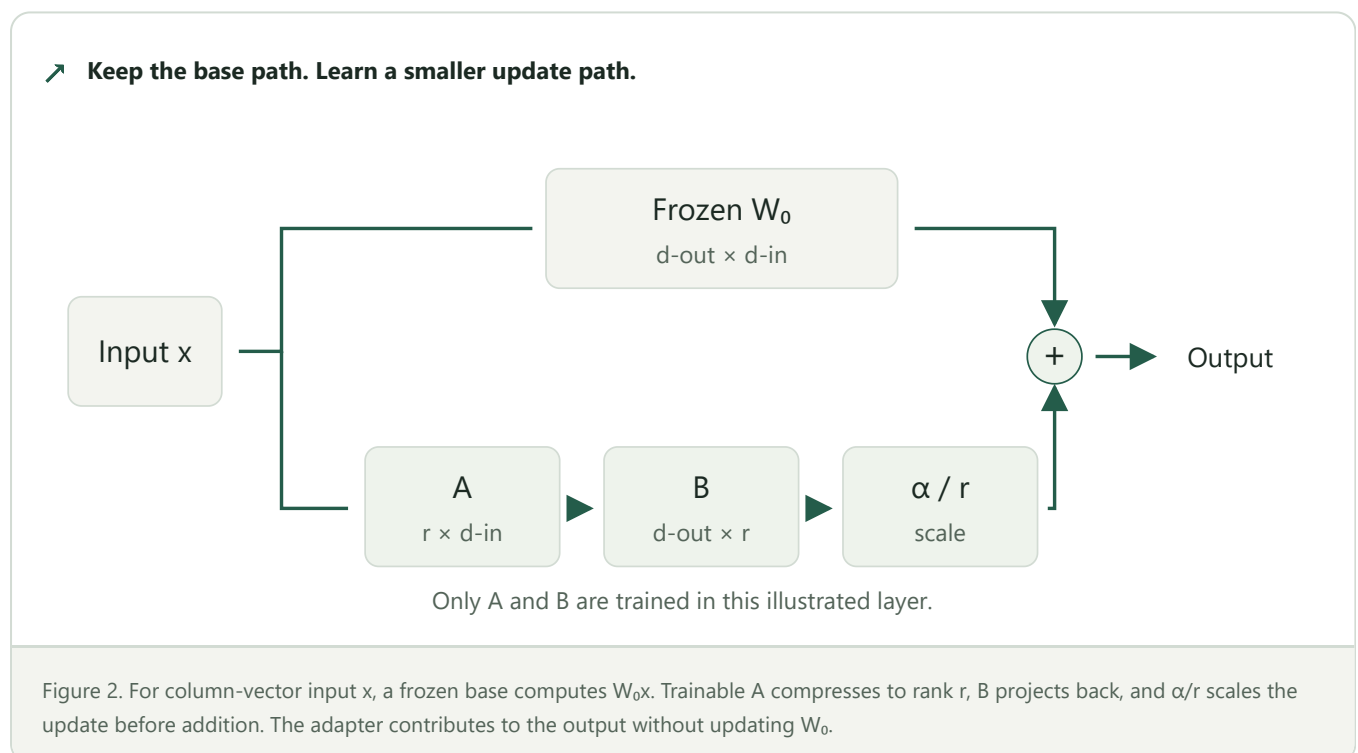
4. Choose which parameters can change

Full fine-tuning updates the base model's weights. **Parameter-efficient fine-tuning (PEFT)** trains a smaller parameter set. These describe parameter choices, while SFT and preference losses describe training objectives; they can be combined.

One PEFT method, **LoRA**, keeps a base weight matrix frozen and learns a low-rank addition. For a column vector x :

$$h = W_0 x + \frac{\alpha}{r} B A x.$$

If W_0 has shape $d_{out} \times d_{in}$, then A is $r \times d_{in}$ and B is $d_{out} \times r$. Their product fits the original layer's shape but has rank at most r . Rank r controls capacity; α controls the update scale. Here we use conventional α/r scaling; other variants exist.



A 4096×4096 matrix has **16,777,216** weights. Rank 8 uses **65,536** adapter parameters, about 0.39% of that count. This compares trainable parameters for one layer, excluding biases and other components. The frozen base still exists and participates in computation.

▼ Go deeper: initialization, rank, and target modules

A common initialization makes one factor nonzero and the other zero, giving a zero initial update. In the lab, A is nonzero and B is zero. B can receive a gradient immediately; A can begin changing after B becomes nonzero. If both factors start at zero, the simple product branch gives zero gradients to both.

Increasing rank increases the update's representational capacity and parameter count. It does not guarantee higher quality. Which layers receive adapters also matters: attention projections and feed-forward projections can affect behavior differently.

For a compatible unquantized linear layer, adding the scaled product to the base matrix can merge the adapter for inference. Preserve the original artifacts and verify the merged model. Quantized merging may involve conversion and rounding, so do not assume bitwise-identical results.

5. Separate quantized storage from learning

QLoRA adapts a model through LoRA while keeping the pretrained base frozen in a quantized representation. Gradients flow through computation using that base into the adapters. The [QLoRA paper](#) combines a 4-bit NormalFloat format, quantized quantization constants, and paged optimizers to reduce memory demands.

It does not mean every multiplication, gradient, activation, and optimizer value is four bits. Storage precision and computation precision differ. Training also needs activations and optimizer state, so the inference memory estimate from Chapter 26 is not a training budget.

Choice	What mainly changes	What remains to verify
Full fine-tuning	Many base parameters	Training memory, regressions, cost
LoRA	Selected low-rank adapter parameters	Rank, target layers, task quality
QLoRA	Adapters around a quantized frozen base	Runtime support, memory peaks, numerical effects

Use validation to choose learning rate, training duration, and adapter settings. Too many epochs can memorize the demonstrations; too few can leave the behavior unchanged. Gradient accumulation combines gradients across microbatches before an optimizer step, allowing a larger effective batch under a memory limit.

6. Turn judgments into preference data

A preference example contains **the same prompt**, two responses, and a judgment about which is better under a rubric. It tells the learner about relative quality, not necessarily the single ideal wording.

For the movie card above, compare:

Response	Judgment under an evidence-based rubric
A: "Moonlight Map fits your limit and calm mood. Availability is not supplied."	Preferred: supported and honest about missing evidence
B: "Guaranteed to be your favorite, streaming on every service tonight!"	Rejected: unsupported certainty and availability claims

Specify what matters: factual support, constraint satisfaction, useful explanation, and suitable length. Collect disagreements and allow ties or invalid pairs during annotation; do not force every ambiguous pair into a confident label. Filter or handle them deliberately in the training recipe.

Human labels are not automatically consistent, and model-generated labels are not independent truth. If judges reward confidence or verbosity over support, optimization can amplify those traits. Preference learning shapes an assistant's response behavior; it is not automatically a recommender trained on each viewer's movie ratings.

▼ Go deeper: preference bias and useful negative examples

Compare replies with the same available evidence. Otherwise, a label can accidentally favor access to additional facts instead of a better response policy. Blind candidate identities and vary their presentation order when collecting judgments.

Easy pairs of excellent versus nonsensical answers may not teach subtle distinctions. Include plausible mistakes such as violating a runtime limit or inventing a platform. Avoid making every rejected answer longer: the learner may discover length as a shortcut.

Record the rubric, annotator or judge provenance, and uncertain cases. Separate disagreements about taste from errors such as contradicted facts. If a judge is another model, audit its outputs against evidence and human review; shared biases can survive a large synthetic dataset.

7. Learn a reward model, then optimize a policy

A common **reinforcement learning from human feedback (RLHF)** workflow begins with SFT, learns a reward model from preference comparisons, then optimizes the response-generating model, called the **policy**. The [InstructGPT](#) paper illustrates this sequence.

A pairwise reward model assigns scores $r(x, y)$. A common comparison model is:

$$P(y^+ \succ y^- \mid x) = \sigma(r(x, y^+) - r(x, y^-)).$$

The sigmoid turns a score difference into a modeled preference probability. Reward scores are learned proxies, not truth scores. The policy generates responses; the reward model scores them; an RL algorithm such as Proximal Policy Optimization (PPO) updates the policy. A reference-policy penalty often discourages excessive drift.

Reward hacking occurs when optimization finds ways to score well without fulfilling the intended goal. Our movie assistant could learn confident praise because the scorer likes it. Evaluate evidence support, constraints, and regressions independently from reward.

➤ Two ways to learn from the same preference pairs.

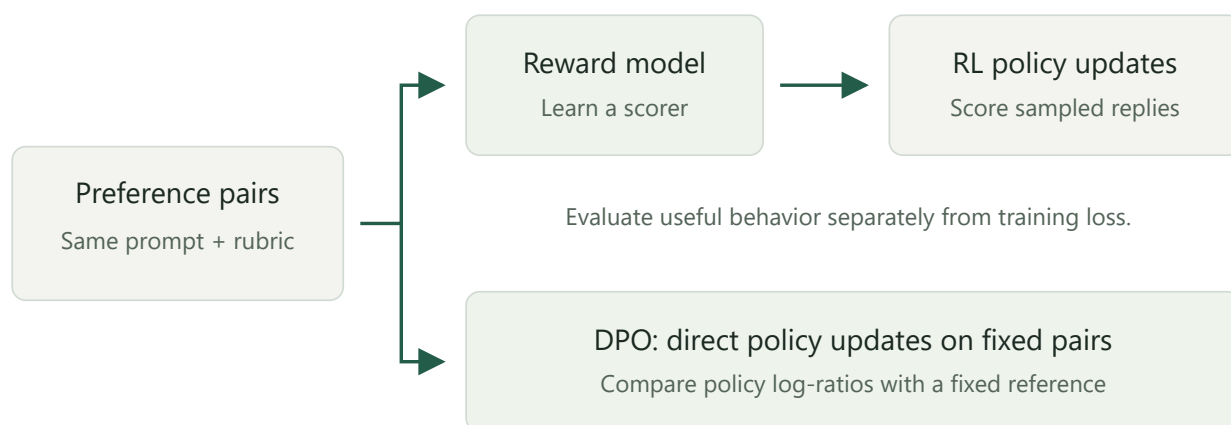


Figure 3. A common RLHF recipe learns a reward model, then optimizes a policy using sampled responses and reference-based regularization. Standard offline DPO instead trains the policy directly on fixed preferred/rejected pairs with a fixed reference. Both need independent evaluation.

▼ Go deeper: KL regularization and online versus offline learning

A simplified regularized objective maximizes expected reward minus $\beta D_{KL}(\pi_\theta \| \pi_{ref})$. KL measures how much the policy's distribution differs from a fixed reference under the evaluated prompts. It is not a distance to truth or a safety certificate.

In an online RL loop, the changing policy generates new responses used for updates. In standard offline DPO, training uses a fixed dataset of response pairs. Iterative approaches can collect new pairs between training rounds; the distinctions concern data collection and the objective, not just names.

PPO constrains policy updates using its own optimization machinery; reference regularization addresses drift from a reference. They are related controls but not interchangeable guarantees. Detailed RL algorithms return later in the handbook.

8. Optimize preferences directly with DPO

Direct Preference Optimization (DPO) trains a policy on chosen/rejected pairs relative to a fixed reference, without fitting a separate reward model in the standard recipe. It is still gradient-based training. See the DPO paper and trainer's loss explanation.

Let $\ell_\theta^+ = \log \pi_\theta(y^+ | x)$ and $\ell_\theta^- = \log \pi_\theta(y^- | x)$, with corresponding reference log-probabilities. With positive scaling coefficient β , define:

$$m = (\ell_\theta^+ - \ell_\theta^-) - (\ell_{ref}^+ - \ell_{ref}^-), \quad L_{DPO} = -\log \sigma(\beta m).$$

The margin measures how much more the policy favors the chosen response over the rejected one **relative to the reference**. Positive margin lowers the pair loss. If policy and reference are identical, the margin is zero and loss is $\log 2 \approx 0.6931$, even if both strongly favor one response already.

For a language model, a completion's log-probability is the sum of its token log-probabilities conditioned on the prompt and preceding completion tokens. Standard DPO compares those completion scores, not a softmax over two displayed sentences. Our lab deliberately reduces the entire output space to two replies so the update can be inspected.

▼ Go deeper: beta, relative likelihood, and a scalar gradient

In DPO's regularized derivation, beta controls a reward-versus-reference tradeoff. In the pair loss, it also scales the margin and gradient. A larger beta is not simply a promise that a short training run will move farther or less far; data, learning rate, and optimization duration interact.

In the lab, $P(A) = \sigma(\theta)$ and $P(B) = 1 - P(A)$, so their log-odds equal θ . For A preferred, $m = \theta - \theta_{ref}$ and:

$$\frac{\partial L}{\partial \theta} = -\beta \sigma(-\beta m), \quad \theta_{new} = \theta - \eta \frac{\partial L}{\partial \theta}.$$

Starting at $\theta = \theta_{ref} = 0$, with $\beta = 0.5$ and learning rate $\eta = 0.5$, the gradient is -0.25 . One update gives $\theta = 0.125$ and $P(A) \approx 0.5312$.

A real vocabulary has many possible continuations. Improving a relative preference does not require both an absolute increase in the chosen response's probability and a decrease in the rejected response's probability. That simple complementary relationship is a special property of our two-outcome lab.

9. Evaluate the behavior and preserve the deployment recipe

Compare the adapted model with the baseline using unseen prompts and the same evidence and tools where possible. Measure factual support, hard constraints, useful style, appropriate uncertainty, and tool-call validity. Use blinded human comparisons or audited judges for subjective quality; record ties and sample counts.

Watch for **overfitting** to demonstrations and **catastrophic forgetting**, where adapting to one distribution harms previous capabilities. Evaluate general tasks as well as the new movie behavior. Falling training loss alongside worsening validation results is a reason to investigate, not to train longer automatically.

Save the base-model identity, tokenizer/chat template, data version, training configuration, adapter or checkpoint, and evaluation report. A LoRA adapter depends on a compatible base architecture and weights. Test the actual deployment artifact, including any merge or quantization, with the intended prompt and generation settings. Keep a tested rollback version.

Training does not grant tool permissions or keep availability current. Continue enforcing access, exact rules, and action validation in the application. A preferred answer can still be wrong; an adapted assistant still needs evidence.

10. Inspect updates and try the experiments

First switch LoRA to its initial state: the adapter contributes zero. Return to the hand-set update, then change rank and alpha. In the preference view, move the steps to zero, take one step, and inspect the gradient. Finally prefer response B: the loss falls even while the unsupported reply becomes more likely.

INSPECT. UPDATE. QUESTION.

Small numerical models · no LLM loaded

Small updates. Different learning signals.

Inspect a low-rank matrix update, then optimize a separate two-response policy.

LoRA example: rank 1 and $\alpha = 2$ give 8 trainable adapter parameters and 16 frozen base parameters. The hand-set update changes output [1, 0.5, 0, 1] to [1.4, 0.5, -0.2, 1.1]. With $B = 0$, the update vanishes.

Preference example: start at reference $P(A) = 0.5$, choose A, $\beta = 0.5$, and $\eta = 0.5$.

Step	θ	P(A)	DPO loss
0	0.0000	0.5000	0.6931
1	0.1250	0.5312	0.6624

If B is incorrectly preferred, the first step gives $\theta = -0.1250$ and $P(A) = 0.4688$, while the loss still falls to 0.6624. Fitting a bad label can reward unsupported claims.

The two panels are separate teaching models. The LoRA panel performs matrix arithmetic using fixed illustrative values. The preference panel genuinely optimizes one scalar over two fixed replies. Neither loads or trains an LLM, measures generalization, or calls an external service. There is no held-out accuracy score because one repeated pair cannot provide one.

The four-file Python 3.10+ workbook uses only the standard library: `data.json`, `tuning.py`, `test_tuning.py`, and `README.md`.

```
py tuning.py
py tuning.py --rank 2 --alpha 4 --stage initial
py tuning.py --steps 1 --chosen A
py tuning.py --steps 40 --chosen B --beta 1
py -m unittest -v test_tuning.py
```

Use `python` or `python3` if needed. Reports contain the matrices and every scalar training step. Invalid configuration choices exit with code 2. Inspect intermediate values instead of interpreting the loss as a quality score.

Exercise 1: identify the masked information

An SFT example excludes user tokens from its loss. Does the assistant lose access to the runtime constraint in the user's request?

▼ Reveal the reasoning

No. The tokens remain in the context. The loss mask determines which prediction positions contribute to the objective; an attention mask determines which positions can attend to which others. The assistant can use the runtime constraint while learning only from its answer tokens.

Exercise 2: count an adapter

A layer has 1024 input and 2048 output features. With rank 4, how many LoRA parameters are trained, excluding bias? Does that count include the base weights?

▼ Reveal the reasoning

A has 4×1024 parameters and B has 2048×4 : 12,288 in total. The frozen base has 2,097,152 weights in addition to the adapter. Training fewer parameters does not remove the base computation or activation memory.

Exercise 3: question a falling preference loss

The labeler prefers the movie response that invents worldwide availability. DPO loss decreases over training. What has improved, and what has not been established?

▼ Reveal the reasoning

The model has better fitted that preference label relative to its reference. Factual correctness has not improved merely because the loss fell; the label rewards a false claim. Correct the rubric and labels, then evaluate held-out evidence support and behavior.

11. Check your understanding

01 What does an assistant-only loss mask do in SFT?

- A. It selects assistant prediction positions for the loss while keeping the prompt as context.
- B. It hides the user request from the model.
- C. It guarantees that every target statement is true.

Answer: A. Loss masking and attention masking have different jobs. The model can attend to the prompt while optimizing the selected target-token predictions.

02 What changes in a standard LoRA-adapted linear layer?

- A. Only the user prompt changes; no parameters are trained.
- B. Low-rank factors are trained while the base matrix stays frozen.
- C. The base model disappears and the adapter runs independently.

Answer: B. LoRA adds a scaled low-rank product to the base path. The base still occupies storage and participates in computation.

03 What does QLoRA imply?

- A. Every gradient and activation is four bits.
- B. The model can train with no activation memory.
- C. Adapters learn through a frozen quantized base, while other tensors can use higher precision.

Answer: C. Quantized weight storage is only one part of the training memory budget. Computation, gradients, activations, and optimizer state have their own requirements.

04 How does standard offline DPO differ from reward-model-based RLHF?

- A. It never updates the policy parameters.
- B. It trains directly on preference pairs relative to a fixed reference without a separate reward-model training stage.
- C. It guarantees factual truth because preferences replace demonstrations.

Answer: B. DPO is gradient-based preference training. Its objective avoids a separately fitted reward model in the standard recipe but still depends on preference-data quality and independent evaluation.

05 A bad label prefers an unsupported movie claim and DPO loss falls. What follows?

- A. The claim must have become true.
- B. The system has passed held-out factuality evaluation.
- C. The model is better fitting that label; useful behavior may have worsened.

Answer: C. The loss rewards the specified comparison. Wrong labels, biased judges, and poor rubrics can steer behavior in an undesirable direction.

12. Connect adaptation to broader generative AI

You can now distinguish the target used for learning, the parameters that change, and the evaluation needed afterward. SFT, LoRA, quantization, and preference optimization are different choices that can appear together in a training recipe.

Chapter 29, **Generative AI beyond language**, broadens the picture to models that generate images and other data. Keep the same questions: what distribution is learned, what objective guides learning, and how do we judge the generated result?

▼ Quick-reference glossary

Term	Meaning
SFT	Supervised fine-tuning on desired outputs
Teacher forcing	Predict target tokens using known earlier target tokens
Loss mask	Selects prediction positions included in the training loss
PEFT	Adaptation that trains a limited parameter set
LoRA	Low-rank update learned around frozen base weights
QLoRA	LoRA adaptation through a quantized frozen base
Policy	Model distribution used to generate responses
Reward model	Learned scorer used as a proxy for desired quality
RLHF	Reinforcement learning guided by human-feedback-derived signals
DPO	Direct preference loss relative to a reference policy
KL divergence	A directional measure of distributional difference
Catastrophic forgetting	Loss of earlier capabilities during adaptation

The linked papers and trainer documentation explain the underlying methods. All movie data and small numerical experiments are illustrative. The reported probabilities are calculated within the toy policy, not measured performance of a language model.